Assignment_6_Data_624

Mubashira Qari

2026-03-22

Table of Contents

9.1. Figure 9.32 shows the ACFs for 36 random numbers, 360 random numbers and 1,000 random numbers.

a. Explain the differences among these figures. Do they all indicate that the data are white noise?

The purpose of an ACF plot is to help us see if there is a hidden pattern in your data. It measures how much an observation is related to its own past values. If the data has a trend or a repeating cycle like seasonality, the ACF plot will show large bars. If the data is just random, the bars will be small and stay within the upper and lower boundaries.

The main difference between the three figures is the width of the blue dashed lines and the size of the individual spikes.

In the plot with 36 numbers, the blue dashed lines are very far apart. As the sample size increases to 360 and then to 1,000, these lines move closer and closer to the center. This happens because the more data we have, the more certain it is about what is random, so it makes the safety zone much narrower.

The individual autocorrelation spikes also tend to look smaller in the larger samples. With only 36 numbers, one or two data points can create a large spike by accident. With 1,000 numbers, those points get pulled by all the other random data, so the spikes naturally stay closer to zero.

b. Why are the critical values at different distances from the mean of zero? Why are the autocorrelations different in each figure when they each refer to white noise?

Why are the critical values at different distances from the mean of zero?

The critical values or those blue dashed lines represent the threshold of safety zone.

With only 36 numbers, we are not very certain about the patterns, so the plot sets a wide safety zone. It allows for larger spikes because they could easily be just a coincidence.

When we have 1,000 numbers, it gets much is much stricter. It expects that if the data is truly random, all those 1,000 numbers should cancel each other out almost perfectly.

Because we have so much data, we are more certain that even a small spike might actually be a real pattern, so the critical values shrink to catch them.

Why are the autocorrelations different in each figure when they each refer to white noise?

ven though all three series are white noise (purely random), the individual spikes look different because of random variation:

In the first plot (36 numbers), one or two numbers that happen to follow each other can create a large autocorrelation spike randomly. There is not enough other data to balance that fluke.

In the largest plot (1,000 numbers), we have so many random data points that any accidental patterns are drowned out. The more random numbers we add, the more they average out toward zero. This is why the spikes in the 1,000 number plot look like tiny vibrations compared to the larger jumps in the 36 number plot.

9.2 A classic example of a non-stationary series are stock prices. Plot the daily closing prices for Amazon stock (contained in gafa_stock), along with the ACF and PACF. Explain how each plot shows that the series is non-stationary and should be differenced.

 # Filtering for Amazon and set a regular index for the plots

```
library(fpp3)

## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr

## ── Attaching packages ──────────────────────────────────────────── fpp3
1.0.2 ──

## ✓ tibble      3.2.1     ✓ tsibble     1.1.6
## ✓ dplyr       1.1.4     ✓ tsibbledata 0.4.1
## ✓ tidyr       1.3.1     ✓ feasts      0.4.2
## ✓ lubridate   1.9.4     ✓ fable       0.5.0
## ✓ ggplot2     4.0.0

## Warning: package 'dplyr' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## Warning: package 'tsibble' was built under R version 4.3.3

## Warning: package 'tsibbledata' was built under R version 4.3.3

## ── Conflicts ──────────────────────────────────────────────────
fpp3_conflicts ──
## ✕ lubridate::date()    masks base::date()
```

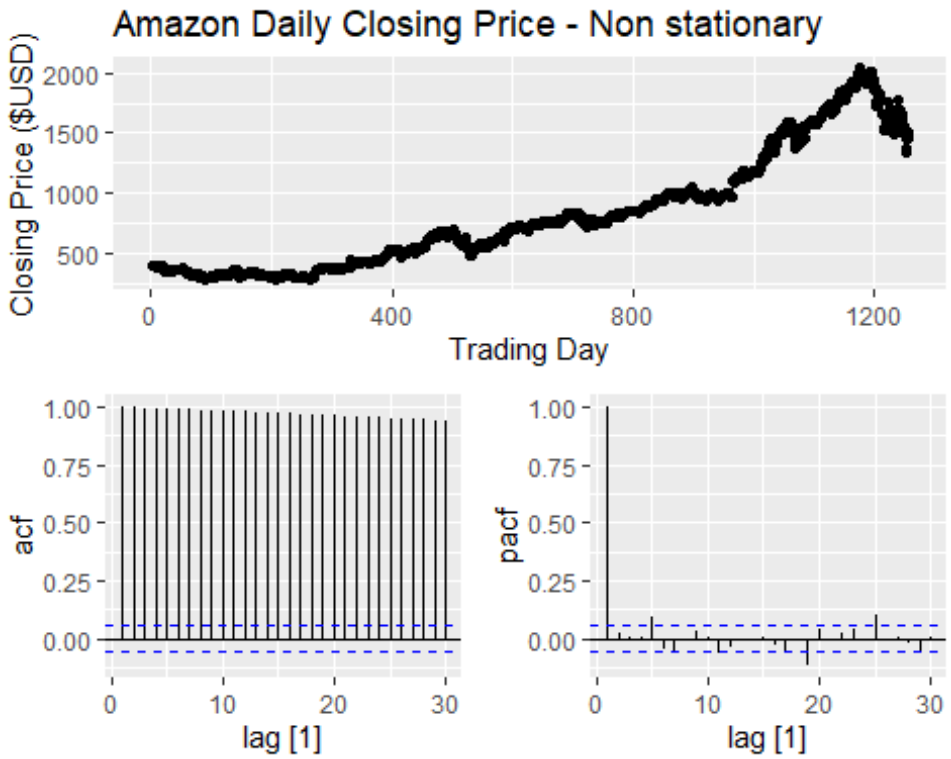
```
## ✕ dplyr::filter()      masks stats::filter()
## ✕ tsibble::intersect() masks base::intersect()
## ✕ tsibble::interval()  masks lubridate::interval()
## ✕ dplyr::lag()         masks stats::lag()
## ✕ tsibble::setdiff()   masks base::setdiff()
## ✕ tsibble::union()     masks base::union()
```

```r
amazon_stock <- gafa_stock |>
  filter(Symbol == "AMZN") |>
  mutate(day = row_number()) |>
  update_tsibble(index = day, regular = TRUE)

#amazon_stock
```

Generating the Time Plot, ACF, and PACF together

```r
amazon_stock |>
  gg_tsdisplay(Close, plot_type = 'partial') +
  labs(title = "Amazon Daily Closing Price - Non stationary",
       y = "Closing Price ($USD)",
       x = "Trading Day")
```

```
## Warning: `gg_tsdisplay()` was deprecated in feasts 0.4.2.
## ℹ Please use `ggtime::gg_tsdisplay()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Reason for Non Stationary Series:

The Trend: can see a very clear, long-term upward trend where the price moves from around $400 to over $2000.

The Mean:Because the price does not hover around a single horizontal line but instead wanders upward, the average value changes depending on which year you look at. This changing mean is the definition of non stationarity.

The ACF Plot (Bottom Left):

The Slow Decay: Notice how the spikes are huge and barely shrink as you move from lag 1 to lag 30.

In a stationary series, these spikes would drop into the safety zone of the blue dashed lines very quickly. This slow, linear decrease is the trend that needs to be removed through differencing.

The PACF Plot (Bottom Right):

There is exactly one massive spike at Lag 1 that hits the 1.0 mark, and almost every other spike is tiny or insignificant.

This tells us that today's price is almost a perfect copy of yesterday's price plus a little bit of random noise. Since the current value is so dependent on the previous value, the series drifts and is non-stationary.

9.3. For the following series, find an appropriate Box-Cox transformation and order of differencing in order to obtain stationary data.

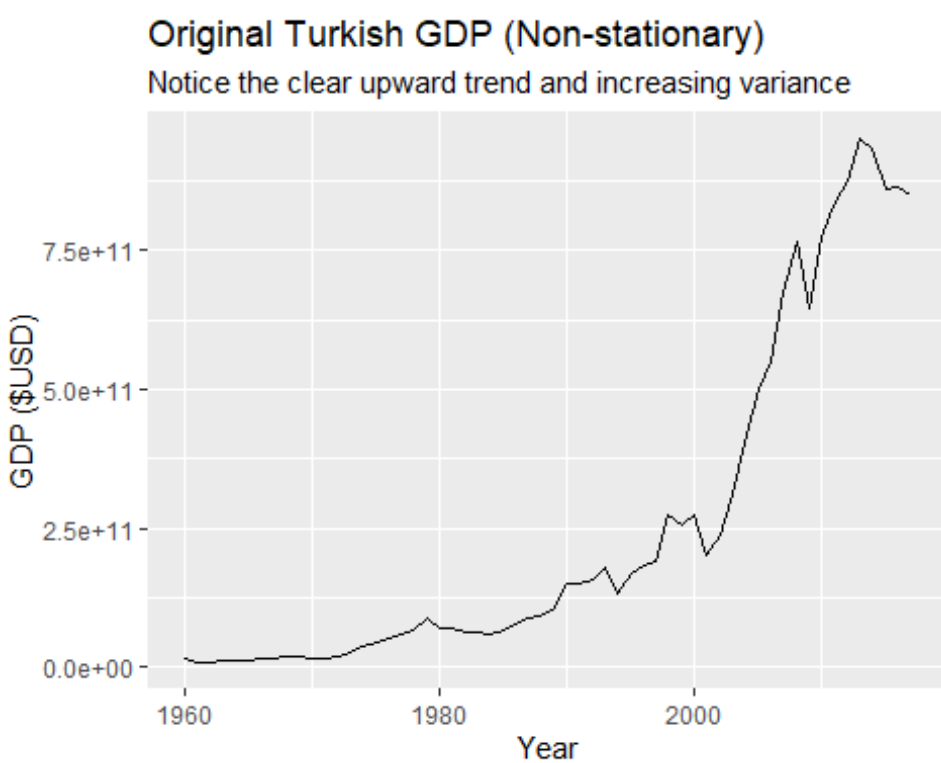
a. Turkish GDP from global_economy.

Prepare the Turkish GDP Data

```
library(fpp3)

turkey_gdp <- global_economy |>
  filter(Country == "Turkey")

#turkey_gdp
```

Plot the original GDP to see the clear upward trend

```
turkey_gdp |>
  autoplot(GDP) +
  labs(title = "Original Turkish GDP (Non-stationary)",
       subtitle = "Notice the clear upward trend and increasing variance",
       y = "GDP ($USD)",
       x = "Year")
```



Finding the optimal Box-Cox transformation (lambda)

```
# This stabilizes the variance so the swings are more even over time.
lambda <- turkey_gdp |>
```

```
  features(GDP, features = guerrero) |>
  pull(lambda_guerrero)

lambda

## [1] 0.1572187
```

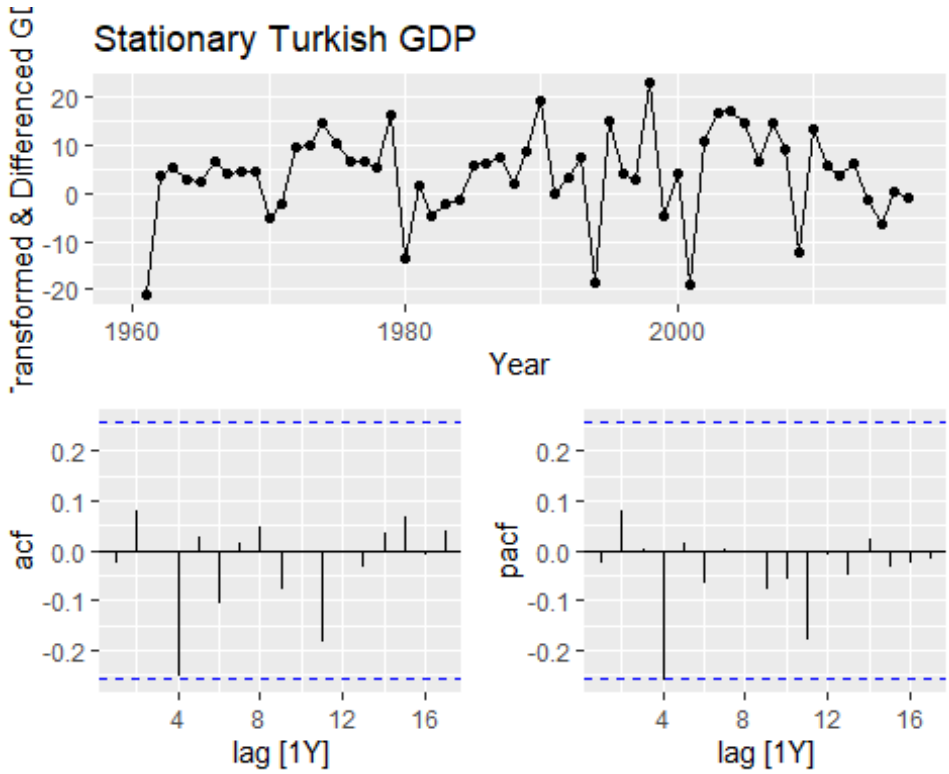
Finding the number of differences needed

Using the KPSS test to see how many times we must difference to reach stationarity.

```
diff_value <- turkey_gdp |>
  mutate(transformed_gdp = box_cox(GDP, lambda)) |>
  features(transformed_gdp, unitroot_ndiffs) |>
  pull(ndiffs)

diff_value

## [1] 1
```

Plotting the final stationary data

```
turkey_gdp |>
  gg_tsdisplay(difference(box_cox(GDP, lambda), differences = diff_value),
               plot_type = 'partial') +
  labs(title = "Stationary Turkish GDP",
       y = "Transformed & Differenced GDP",
       x = "Year")

## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_point()`).
```



Interpretation

To make the Turkish GDP data stationary, we applied a Box-Cox transformation with lambda = 0.157 and one first difference d = 1.

We have achieved Stabilized Variance: Before the transformation, the GDP fluctuations likely increased as the economy grew. This specific lambda value supressed those swings, ensuring the variation is consistent across the entire time period.

Removed Trend d = 1: The original data had a clear upward trend.

By taking one difference, we shifted the focus from total GDP to the year-to-year change, which flattened the series.

Stationary Result: In the final plots, the time series now bounces randomly around a constant mean of zero.

Both the ACF and PACF plots show that nearly all vertical spikes stay within the critical values - the blue dashed lines.

This confirms that the remaining data is essentially random noise.

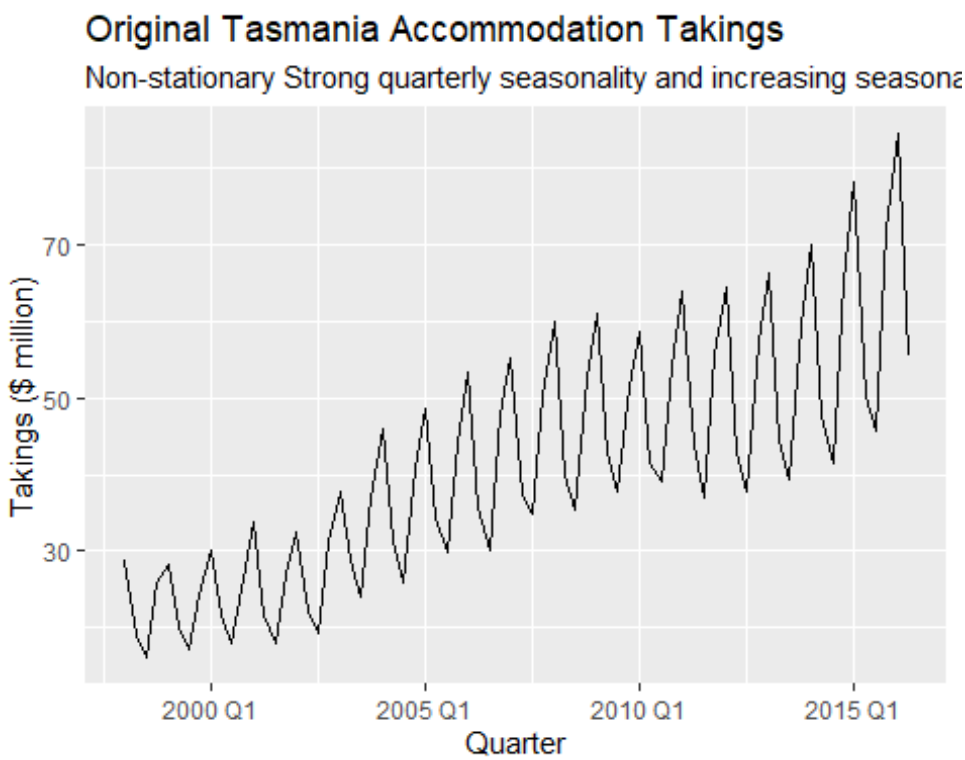
Accommodation takings in the state of Tasmania from aus_accommodation.

Prepare the data for Tasmania

```r
tasmania_data <- aus_accommodation |>
  filter(State == "Tasmania")

# tasmania_data
```

Plotting Original Data Plotting original Tasmania takings to show seasonality and growth

```r
aus_accommodation |>
  filter(State == "Tasmania") |>
  autoplot(Takings) +
  labs(title = "Original Tasmania Accommodation Takings",
       subtitle = "Non-stationary Strong quarterly seasonality and increasing
seasonal swings",
       y = "Takings ($ million)",
       x = "Quarter")
```




```r
aus_accommodation

## # A tsibble: 592 x 5 [1Q]
## # Key:       State [8]
##       Date State                        Takings Occupancy   CPI
##      <qtr> <chr>                         <dbl>     <dbl> <dbl>
##  1 1998 Q1 Australian Capital Territory    24.3      65    67
##  2 1998 Q2 Australian Capital Territory    22.3      59    67.4
##  3 1998 Q3 Australian Capital Territory    22.5      58    67.5
##  4 1998 Q4 Australian Capital Territory    24.4      59    67.8
##  5 1999 Q1 Australian Capital Territory    23.7      58    67.8
```

```
##  6 1999 Q2 Australian Capital Territory    25.4      61    68.1
##  7 1999 Q3 Australian Capital Territory    28.2      66    68.7
##  8 1999 Q4 Australian Capital Territory    25.8      60    69.1
##  9 2000 Q1 Australian Capital Territory    27.3      60.9  69.7
## 10 2000 Q2 Australian Capital Territory    30.1      64.7  70.2
## # ℹ 582 more rows
```

Finding the optimal Box-Cox transformation (lambda)

For handling the increasing seasonal variation.

```
lambda_tas <- tasmania_data |>
  features(Takings, features = guerrero) |>
  pull(lambda_guerrero)

lambda_tas

## [1] 0.001819643
```

Determine the order of differencing (d)

Because this is quarterly data, we check for seasonal and regular differencing.

```
tas_transformed <- tasmania_data |>
  mutate(trans_takings = box_cox(Takings, lambda_tas))

#tas_transformed
```

Checking for seasonal differencing (D)

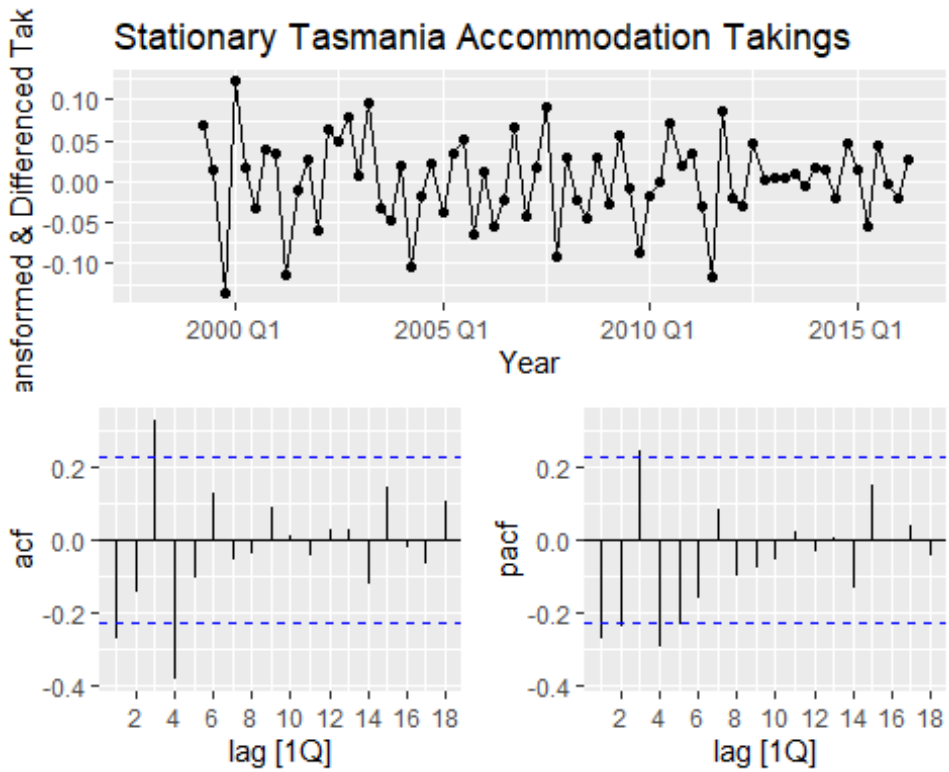
```
Diff_value <- tas_transformed |>
  features(trans_takings, unitroot_nsdiffs) |>
  pull(nsdiffs)

Diff_value

## [1] 1
```

Check for regular differencing (d) after seasonal difference

```
d_value <- tas_transformed |>
  mutate(diff_takings = difference(trans_takings, lag = 4 * Diff_value)) |>
  features(diff_takings, unitroot_ndiffs) |>
  pull(ndiffs)

d_value

## [1] 0
```

Plotting the final stationary data

```r
tasmania_data |>
  gg_tsdisplay(difference(box_cox(Takings, lambda_tas), lag = 4) |>
                 difference(),
               plot_type = 'partial') +
labs(title = "Stationary Tasmania Accommodation Takings",
       y = "Transformed & Differenced Takings",
       x = "Year")

## Warning: Removed 5 rows containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 5 rows containing missing values or values outside the
scale range
## (`geom_point()`).
```



With a lambda value so close to zero (0.0018),

We have effectively applied a log transformation to the Tasmania accommodation data.

Combined with differencing, the resulting plots provide the White Noise Evidence needed for stationarity.

Interpretation of Results

Stabilized Seasonal Variation (lambda = 0.0018): I used this transformation because the seasonal swings in the original Tasmania data grew larger as the total takings increased over time.

This near-zero lambda squashes those growing peaks, making the variation consistent throughout the entire series.

Removed Seasonality and Trend (D=1, d=0): By using one seasonal difference (D=1), I removed the predictable quarterly patterns e.g., summer peaks.

Since my d_value is 0, it means that once the seasonality was removed, the mean became constant enough that no further regular differencing was required to flatten the series.

Stationary Result: In my final ACF and PACF plots for Tasmania, the large, slow-decaying spikes have disappeared. Almost all vertical bars now stay within the blue dashed critical values, proving the data is now stationary.

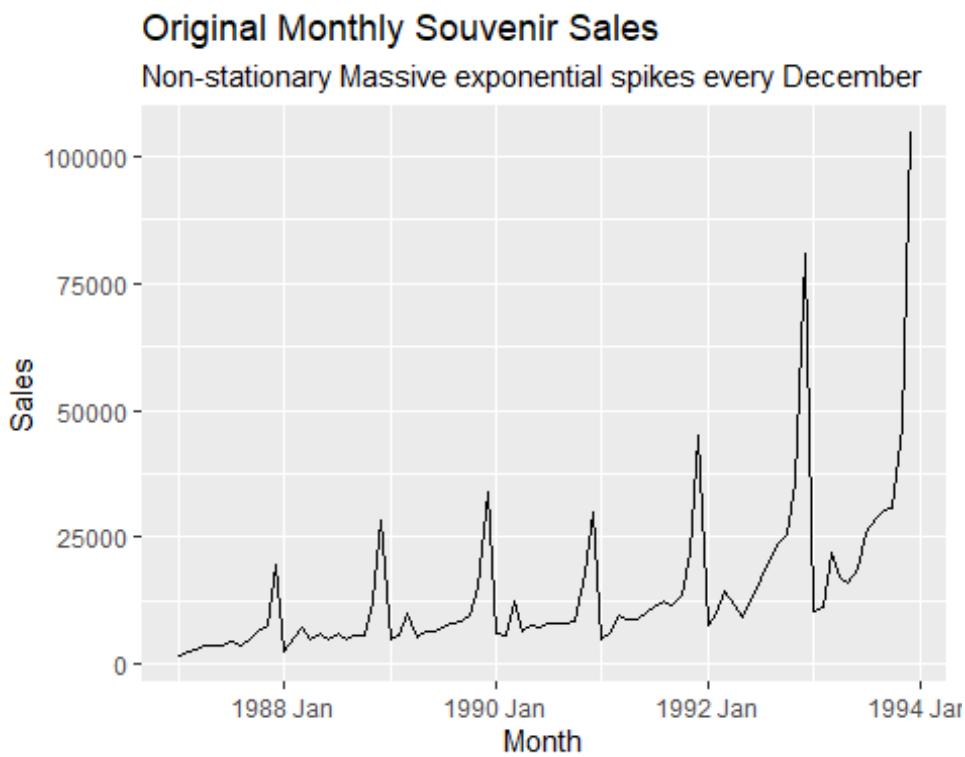
Monthly sales from souvenirs.

Prepare the data

```
souvenir_sales <- souvenirs

# souvenir_sales
```

Plotting Original Data Plotting original souvenir sales to show exponential seasonal peaks

```
souvenirs |>
  autoplot(Sales) +
  labs(title = "Original Monthly Souvenir Sales",
       subtitle = "Non-stationary Massive exponential spikes every December",
       y = "Sales",
       x = "Month")
```



Finding the optimal Box-Cox transformation (lambda) This is important since the Christmas peaks grow exponentially.

```
lambda_souvenir <- souvenir_sales |>
  features(Sales, features = guerrero) |>
  pull(lambda_guerrero)



lambda_souvenir

## [1] 0.002118221
```

**Finding the order of differencing and Checking for seasonal differencing (D) first (lag 12 for monthly)

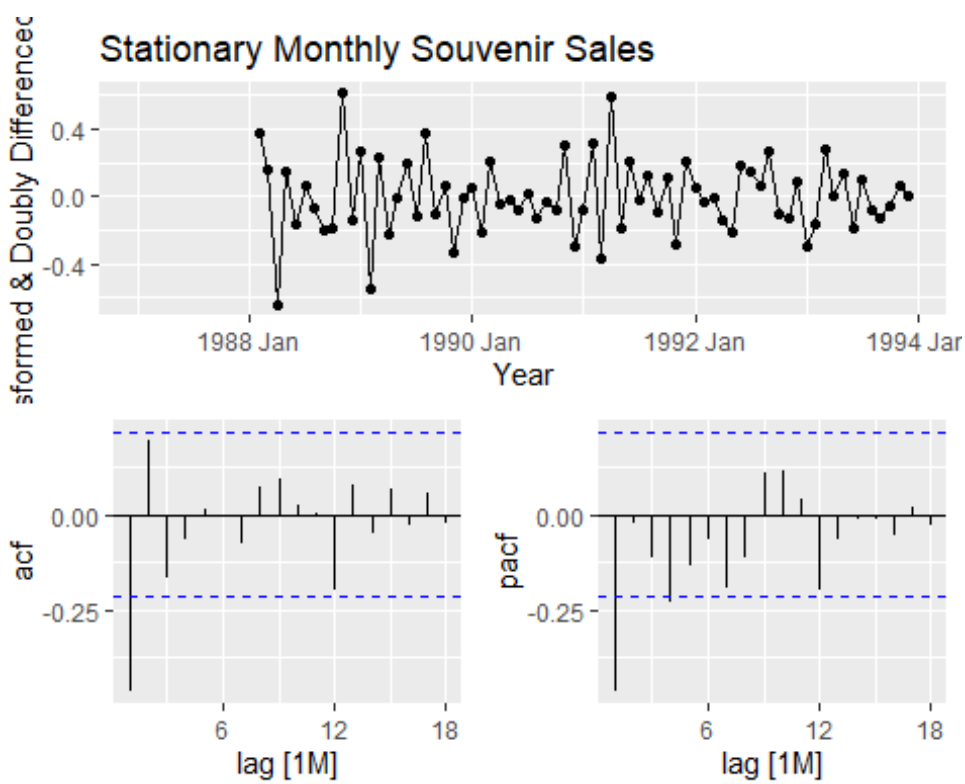
```
Diff_souvenir <- souvenir_sales |>
  mutate(trans_sales = box_cox(Sales, lambda_souvenir)) |>
  features(trans_sales, unitroot_nsdiffs) |>
  pull(nsdiffs)

Diff_souvenir

## [1] 1
```

Check for regular differencing (d) after seasonal difference

```
d_souvenir <- souvenir_sales |>
  mutate(trans_sales = box_cox(Sales, lambda_souvenir)) |>
  mutate(diff_sales = difference(trans_sales, lag = 12 * Diff_souvenir)) |>
  features(diff_sales, unitroot_ndiffs) |>
  pull(ndiffs)
d_souvenir

## [1] 0
```

** Plot the final stationary data**

```
souvenir_sales |>
  gg_tsdisplay(difference(box_cox(Sales, lambda_souvenir), lag = 12) |>
                 difference(),
               plot_type = 'partial') +
  labs(title = "Stationary Monthly Souvenir Sales",
       y = "Transformed & Doubly Differenced Sales",
       x = "Year")

## Warning: Removed 13 rows containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 13 rows containing missing values or values outside the
scale range
## (`geom_point()`).
```



Monthly Souvenir SalesStabilized Peaks (lambda = 0.0021): I utilized a near-zero lambda to tame the massive

exponential growth seen in the December sales spikes, making the yearly fluctuations uniform throughout the series.

Removed Strong Seasonality (D = 1): I used one seasonal difference (lag = 12) to eliminate the predictable monthly cycles, specifically the massive jump every December.

Stationary Result: My final plots confirm that both the long-term trends and seasonal waves have disappeared and the data is now centered at zero.

9.5. For your retail data (from Exercise 7 in Section 2.10), find the appropriate order of differencing (after transformation if necessary) to obtain stationary data.

```
set.seed(98765432)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))
# myseries
```

ind the optimal Box-Cox transformation (lambda) Retail data often shows increasing seasonal variation with higher sales.

```
lambda_retail <- myseries |>
  features(Turnover, features = guerrero) |>
  pull(lambda_guerrero)

lambda_retail

## [1] 0.05068016
```

Finding the order of differencing Checking for seasonal differencing (D) first (lag 12 for monthly retail data)

```
Diff_retail <- myseries |>
  mutate(trans_turnover = box_cox(Turnover, lambda_retail)) |>
  features(trans_turnover, unitroot_nsdiffs) |>
  pull(nsdiffs)

Diff_retail

## [1] 1
```

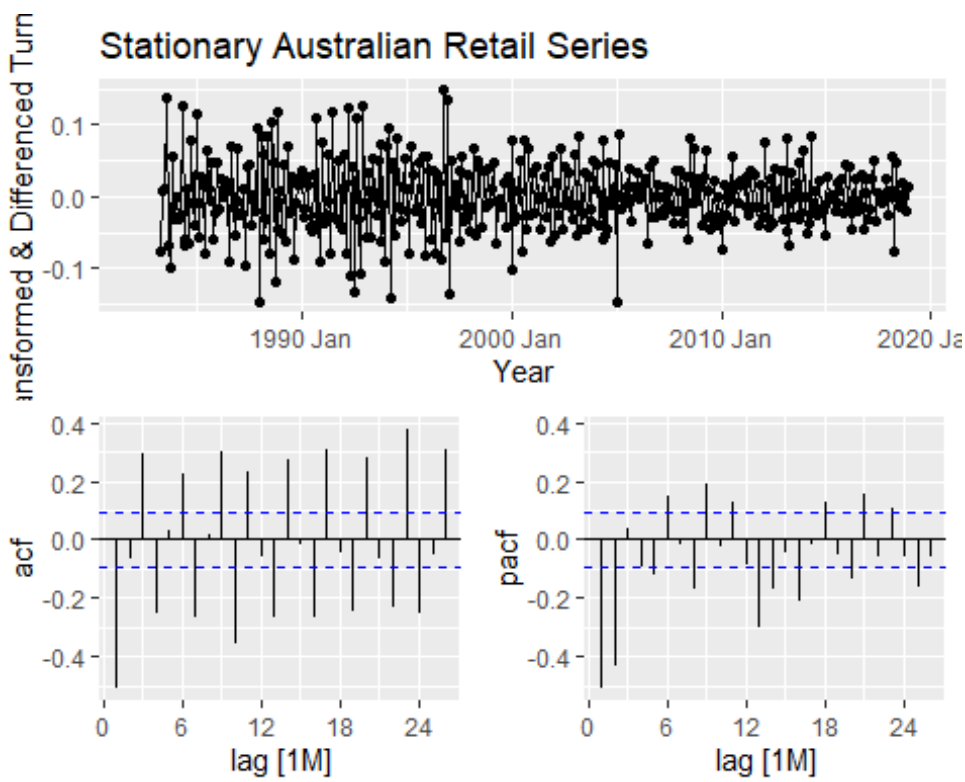
**Check for regular differencing (d) after seasonal difference

```
d_retail <- myseries |>
  mutate(trans_turnover = box_cox(Turnover, lambda_retail)) |>
  mutate(diff_turnover = difference(trans_turnover, lag = 12 * Diff_retail))
|>
  features(diff_turnover, unitroot_ndiffs) |>
  pull(ndiffs)
```

```
# Display the result
d_retail

## [1] 1
```

Plot the final stationary data

```
myseries |>
  gg_tsdisplay(difference(box_cox(Turnover, lambda_retail), lag = 12 *
Diff_retail) |>
                 difference(differences = d_retail),
               plot_type = 'partial') +
  labs(title = "Stationary Australian Retail Series",
       y = "Transformed & Differenced Turnover",
       x = "Year")

## Warning: Removed 13 rows containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 13 rows containing missing values or values outside the
scale range
## (`geom_point()`).
```



Interpretation:

Australian Retail Series Stationarity: The time plot is centered at zero, but the ACF and PACF plots show persistent, significant spikes at seasonal lags (12, 24).

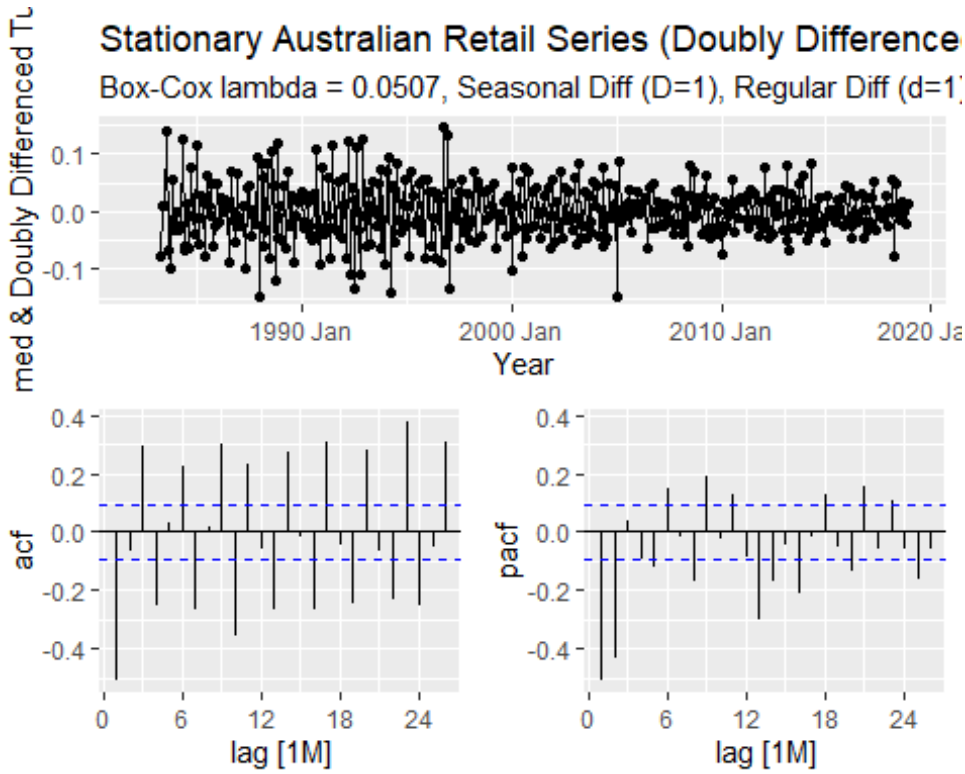
This indicates that even after the transformation (lambda = 0.05) and regular differencing (d=1), there is still significant remaining seasonality.

To achieve true stationarity for this specific series, a seasonal difference (D=1) is needed in addition to the regular difference.

Applying the double difference (D=1, d=1) to my selected series

```
myseries |>
  gg_tsdisplay(
    difference(
      box_cox(Turnover, lambda_retail),
      lag = 12
    ) |>
    difference(),
    plot_type = 'partial'
  ) +
  labs(
    title = "Stationary Australian Retail Series (Doubly Differenced)",
    subtitle = "Box-Cox lambda = 0.0507, Seasonal Diff (D=1), Regular Diff
(d=1)",
    y = "Transformed & Doubly Differenced Turnover",
    x = "Year"
  )
```

```
## Warning: Removed 13 rows containing missing values or values outside the
scale range
## (`geom_line()`).

## Warning: Removed 13 rows containing missing values or values outside the
scale range
## (`geom_point()`).
```



Interpretation

Even though I applied two types of differencing, the spikes crossing the blue lines show that some predictable patterns still remain in the data, which I would eventually fix by choosing specific ARIMA model settings to capture that leftover information.

9.6. Simulate and plot some data from simple ARIMA models.

a. Use the following R code to generate data from an AR(1) model

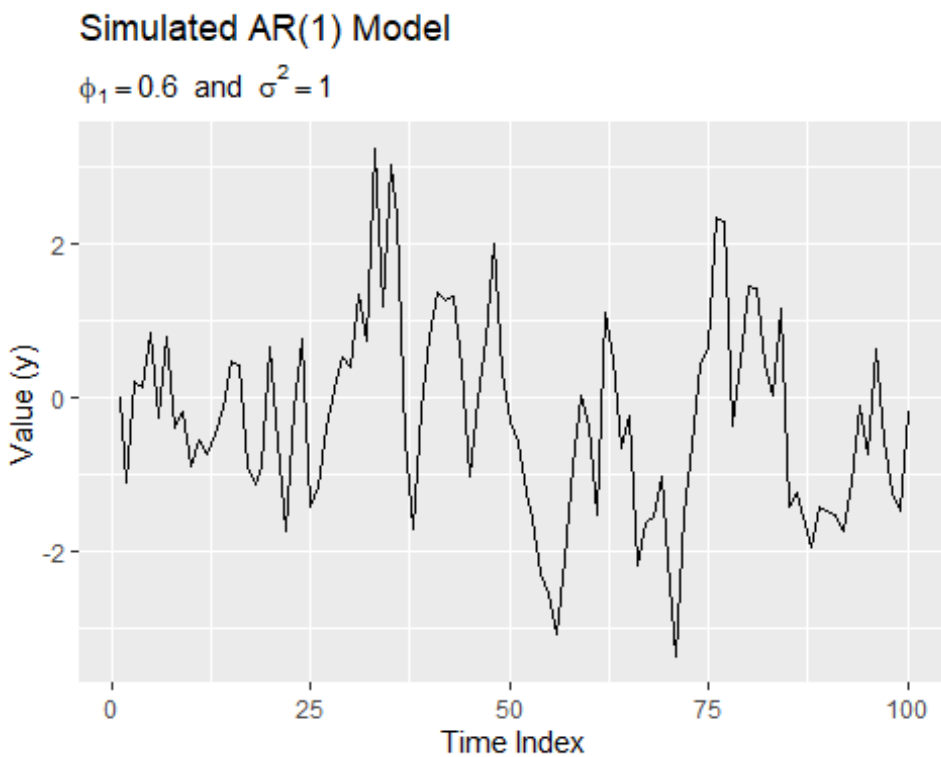
Simulate the AR(1) process

```
# sigma^2 = 1 is the default for rnorm
y <- numeric(100)
e <- rnorm(100)
for(i in 2:100) {
  y[i] <- 0.6 * y[i-1] + e[i]
}
```

Convert to a tsibble for plotting the simulated data

```
# Creating 'sim_ar1'
sim_ar1 <- tsibble(idx = seq_len(100), y = y, index = idx)

# Plotting
sim_ar1 |>
  autoplot(y) +
  labs(title = "Simulated AR(1) Model",
       subtitle = expression(phi[1] == 0.6~~and~~sigma^2 == 1),
```

```
       x = "Time Index",
       y = "Value (y)")
```

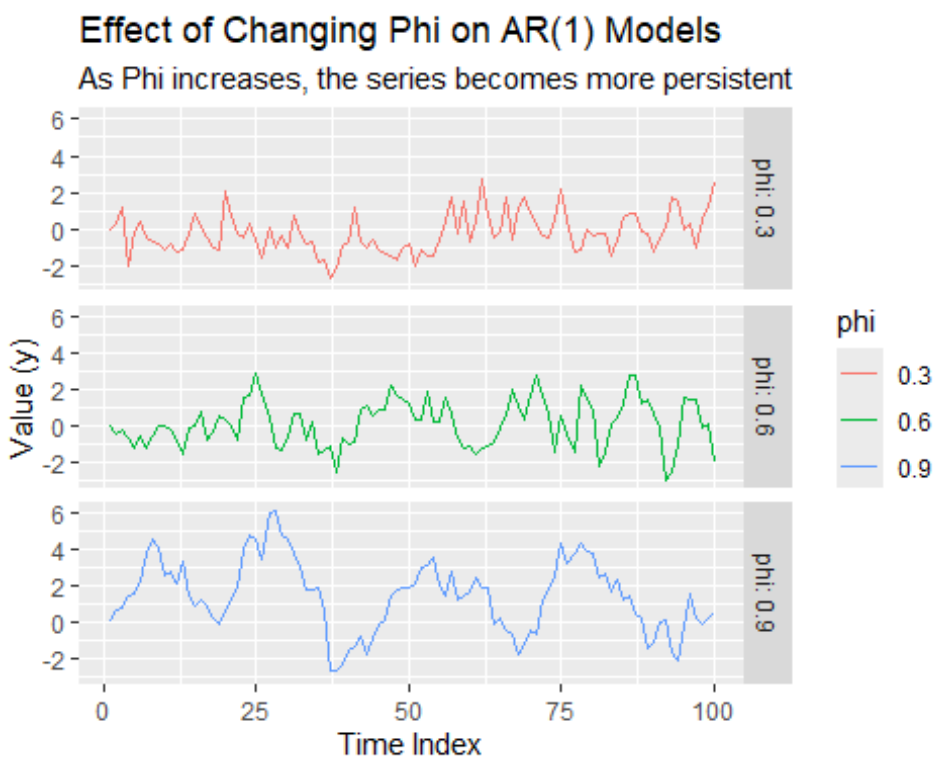


I would describe this as a stationary process with high positive autocorrelation. It is stationary because its statistical properties (mean and variance) do not change over time, even though the 0.6 value results in more momentum, which creates the illusion of a trend compared to the purely random fluctuations of white noise.

b. Produce a time plot for the series. How does the plot change as you change...

```
library(fpp3)
set.seed(1234)

# Function to simulate AR(1) for a given phi
sim_ar1 <- function(phi, n = 100) {
  y <- numeric(n)
  e <- rnorm(n)
  for(i in 2:n) y[i] <- phi * y[i-1] + e[i]
  return(y)
}

# Generating three different scenarios
compare_ar1 <- tsibble(
  idx = rep(1:100, 3),
  phi = rep(c(0.3, 0.6, 0.9), each = 100),
  y = c(sim_ar1(0.3), sim_ar1(0.6), sim_ar1(0.9)),
```

```r
  index = idx,
  key = phi
)

# Plotting the comparison
compare_ar1 |>
  autoplot(y) +
  facet_grid(phi ~ ., labeller = label_both) +
  labs(title = "Effect of Changing Phi on AR(1) Models",
       subtitle = "As Phi increases, the series becomes more persistent",
       x = "Time Index",
       y = "Value (y)")
```



Interpretation

phi(Autoregressive (AR) coefficient):

It tells you how much of yesterday's value carries over into today.

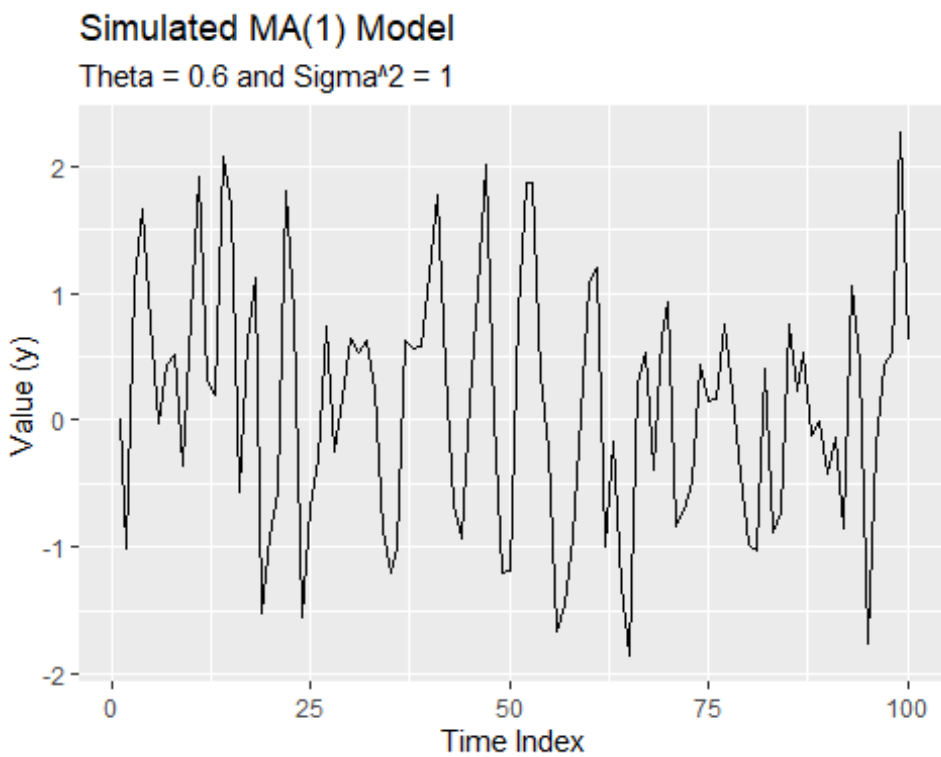
A value of 0.3 means the series has a weak memory. Only 30% of the previous value affects the current one, while the other 70% is just random noise.

The result: Because the memory is low, the data doesn't stay in one place for long. It looks very similar to random white noise because it is constantly jumping back toward the average of zero.

c. Write your own code to generate data from an MA(1) model...

Generating data from a Moving Average (MA(1)) model where the shift parameter is 0.6 and the variance is 1, Unlike the Autoregressive (AR) model which depends on its own past values, this model depends on the current and previous random error terms.

```r
library(fpp3)

# Setting seed for reproducibility
set.seed(87654321)

# Defining parameters: theta = 0.6, sigma^2 = 1
n <- 100
e <- rnorm(n, mean = 0, sd = 1) # White noise errors
y <- numeric(n)

# Generating MA(1) data: y_t = e_t + theta_1 * e_{t-1}
for(i in 2:n) {
  y[i] <- e[i] + 0.6 * e[i-1]
}

# Converting to tsibble and plot
sim_ma1 <- tsibble(idx = seq_len(n), y = y, index = idx)

sim_ma1 |>
  autoplot(y) +
  labs(title = "Simulated MA(1) Model",
       subtitle = "Theta = 0.6 and Sigma^2 = 1",
       x = "Time Index",
       y = "Value (y)")
```



Interpretation

Unlike the AR(1) plots, this series only remembers the very last random shock.

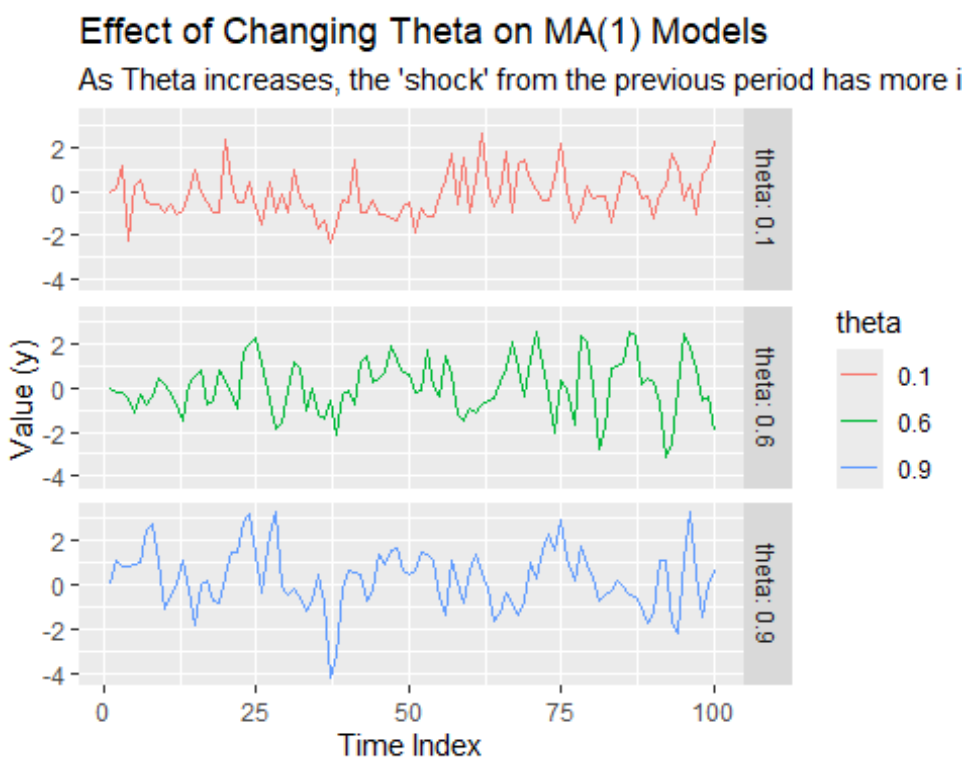
Because the memory is so short like exactly one period, the data never stays above or below the zero line for long and it is constantly pulled back to the middle.

Pure Stationarity: The series is perfectly stationary because its average (0) and its spread (variance) stay the same from the beginning of the index to the end.

d. Produce a time plot for the series. How does the plot change as you change...

```
library(fpp3)
set.seed(1234)

# Function to simulate MA(1) for a given theta
sim_ma1 <- function(theta, n = 100) {
  y <- numeric(n)
  e <- rnorm(n)
  for(i in 2:n) y[i] <- e[i] + theta * e[i-1]
  return(y)
}

# Generating three different scenarios
compare_ma1 <- tsibble(
  idx = rep(1:100, 3),
  theta = rep(c(0.1, 0.6, 0.9), each = 100),
```

```
  y = c(sim_ma1(0.1), sim_ma1(0.6), sim_ma1(0.9)),
  index = idx,
  key = theta
)

# Plotting the comparison
compare_ma1 |>
  autoplot(y) +
  facet_grid(theta ~ ., labeller = label_both) +
  labs(title = "Effect of Changing Theta on MA(1) Models",
       subtitle = "As Theta increases, the 'shock' from the previous period
has more influence",
       x = "Time Index",
       y = "Value (y)")
```



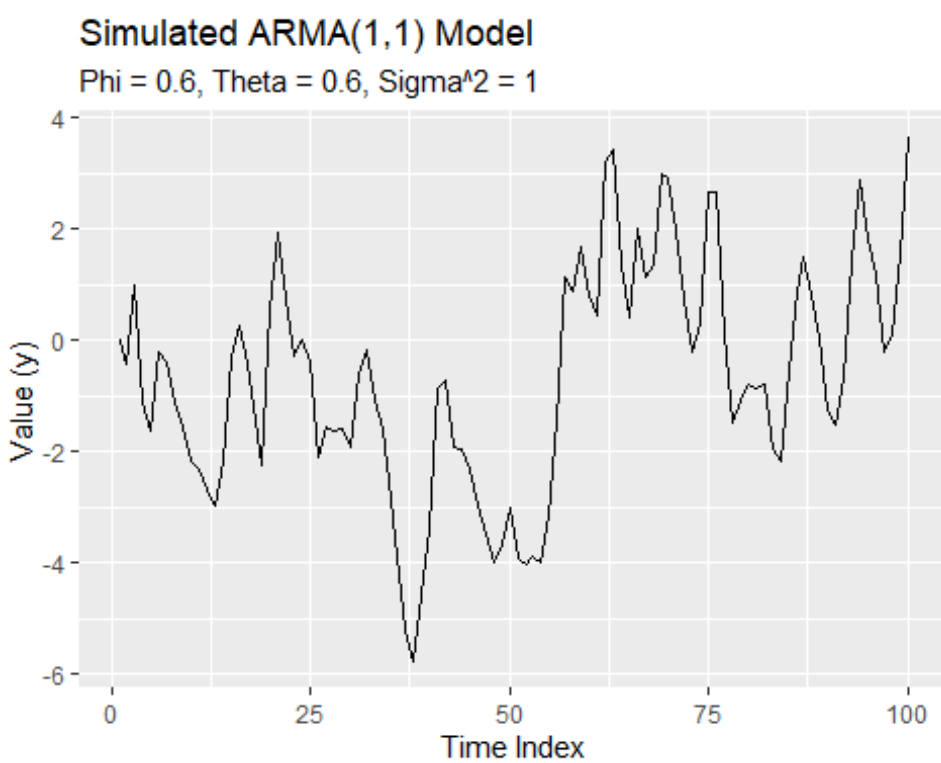
Low Coefficient (0.1): The series looks like pure random noise because the impact of the previous shock is almost zero.

Moderate Coefficient (0.6): The plot becomes slightly smoother in the short term, as more of yesterday's random error carries over into today.

High Coefficient (0.9): The connection between neighboring points is most obvious, but the series never develops long waves and remains choppy because it forgets everything after exactly one time step.

e. Generate data from an ARMA(1,1) model with...

```r
library(fpp3)

#  Setting seed for consistency
set.seed(1234)

#  Defining parameters: phi = 0.6, theta = 0.6, sigma^2 = 1
n <- 100
e <- rnorm(n, mean = 0, sd = 1)
y <- numeric(n)

# Generating ARMA(1,1) data: y_t = phi*y_{t-1} + e_t + theta*e_{t-1}
for(i in 2:n) {
  y[i] <- 0.6 * y[i-1] + e[i] + 0.6 * e[i-1]
}

#  Converting to tsibble and plot
sim_arma11 <- tsibble(idx = seq_len(n), y = y, index = idx)

sim_arma11 |>
  autoplot(y) +
  labs(title = "Simulated ARMA(1,1) Model",
       subtitle = "Phi = 0.6, Theta = 0.6, Sigma^2 = 1",
       x = "Time Index",
       y = "Value (y)")
```

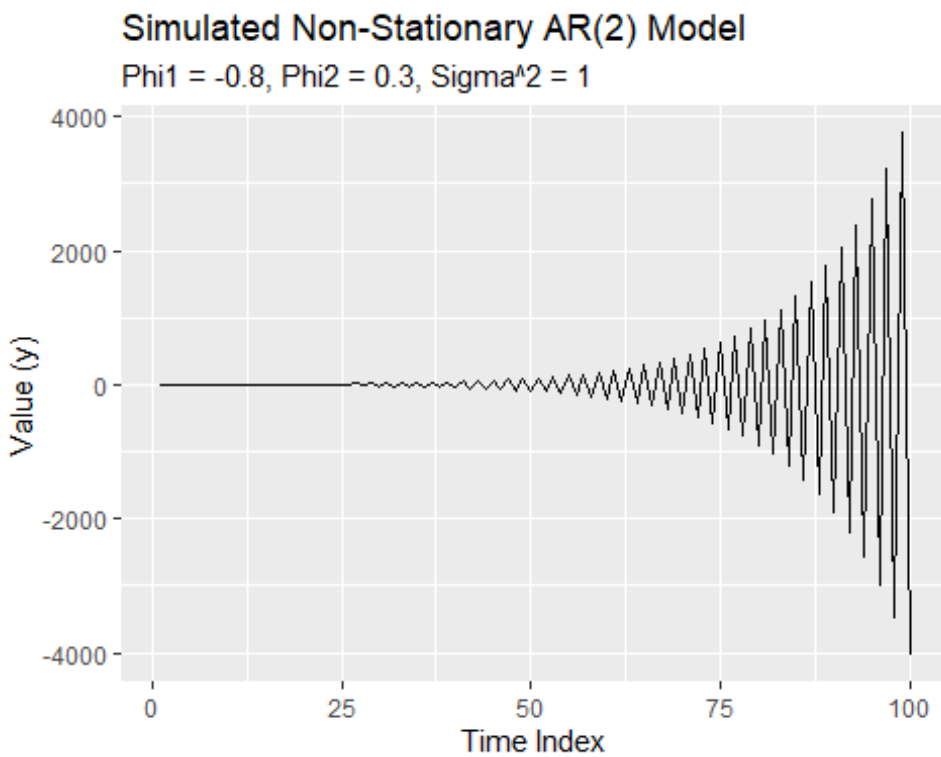


Interpretation

This chart combines long-lasting waves with extra short-term jitters. It is stationary because it always returns to zero, but it looks busier than the others because it uses both a memory score and a shock score at the same time.

f. Generate data from an AR(2) model with...

```r
library(fpp3)

# Setting seed for consistency
set.seed(1234)

# Defining parameters and generate AR(2) data
# Current value = (-0.8 * last value) + (0.3 * value before that) + error
n <- 100
e <- rnorm(n)
y <- numeric(n)

for(i in 3:n) {
  y[i] <- -0.8 * y[i-1] + 0.3 * y[i-2] + e[i]
}

# Converting to tsibble and plot
sim_ar2 <- tsibble(idx = seq_len(n), y = y, index = idx)

sim_ar2 |>
  autoplot(y) +
  labs(title = "Simulated Non-Stationary AR(2) Model",
       subtitle = "Phi1 = -0.8, Phi2 = 0.3, Sigma^2 = 1",
       x = "Time Index",
       y = "Value (y)")
```



**Interpretation* Unlike all the previous stationary charts that stayed near zero, this data explodes as time moves forward.

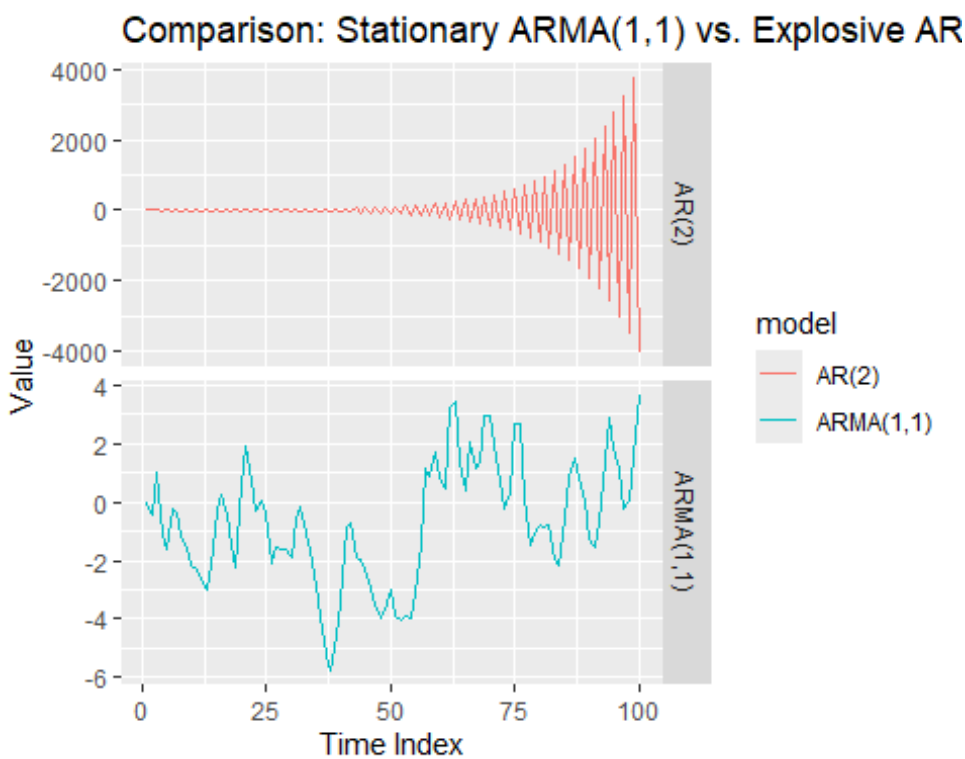
The data oscillates (jumps up and down) with increasing intensity, eventually reaching values as high as 4,000 and as low as -4,000.

Non-Stationary: This is a perfect example of a non-stationary series because its spread (variance) is not constant; it grows larger and larger the longer the process runs.

g. Graph the latter two series and compare them.

```
library(fpp3)
set.seed(1234)

# Generating ARMA(1,1) data (Stationary)
n <- 100
e <- rnorm(n)
y_arma <- numeric(n)
for(i in 2:n) y_arma[i] <- 0.6 * y_arma[i-1] + e[i] + 0.6 * e[i-1]

# Generating AR(2) data (Non-Stationary)
y_ar2 <- numeric(n)
for(i in 3:n) y_ar2[i] <- -0.8 * y_ar2[i-1] + 0.3 * y_ar2[i-2] + e[i]

# Combining and plot
compare_final <- tsibble(
  idx = rep(1:n, 2),
```

```r
  model = rep(c("ARMA(1,1)", "AR(2)"), each = n),
  value = c(y_arma, y_ar2),
  index = idx,
  key = model
)

compare_final |>
  autoplot(value) +
  facet_grid(model ~ ., scales = "free_y") +
  labs(title = "Comparison: Stationary ARMA(1,1) vs. Explosive AR(2)",
       x = "Time Index",
       y = "Value")
```



Interpretation

The ARMA(1,1) model is stable and stationary, while the AR(2) model is explosive and non-stationary.

Notice the Y-axis. The stable model stays between -6 and 4, while the explosive model wildly swings between -4,000 and 4,000.

The stable model always pulls back to zero, making it predictable for forecasting. The explosive model drifts further away the longer it runs, making it impossible to predict.

9.7. Consider aus_airpassengers, the total number of passengers (in millions) from Australian air carriers for the period 1970-2011.

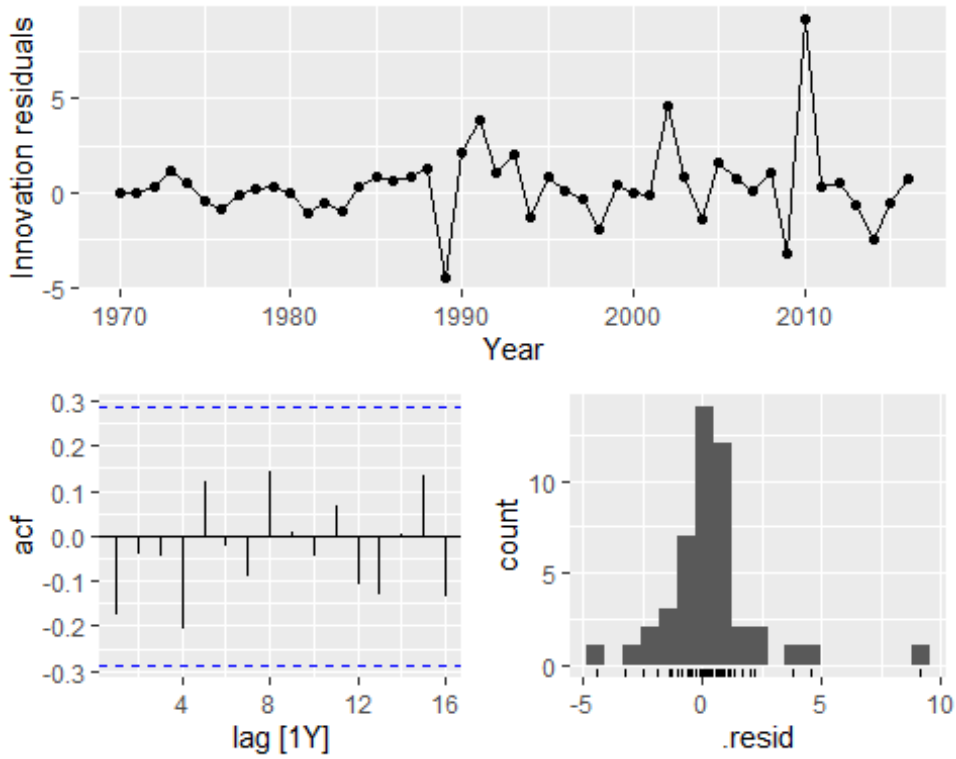
a. Use ARIMA() to find an appropriate ARIMA model. What model was selected. Check that the residuals look like white noise. Plot forecasts for the next 10 periods.

```r
library(fpp3)

# Fitting the automated ARIMA model
aus_fit <- aus_airpassengers |>
  model(ARIMA(Passengers))

# Displaying the selected model
report(aus_fit)
```

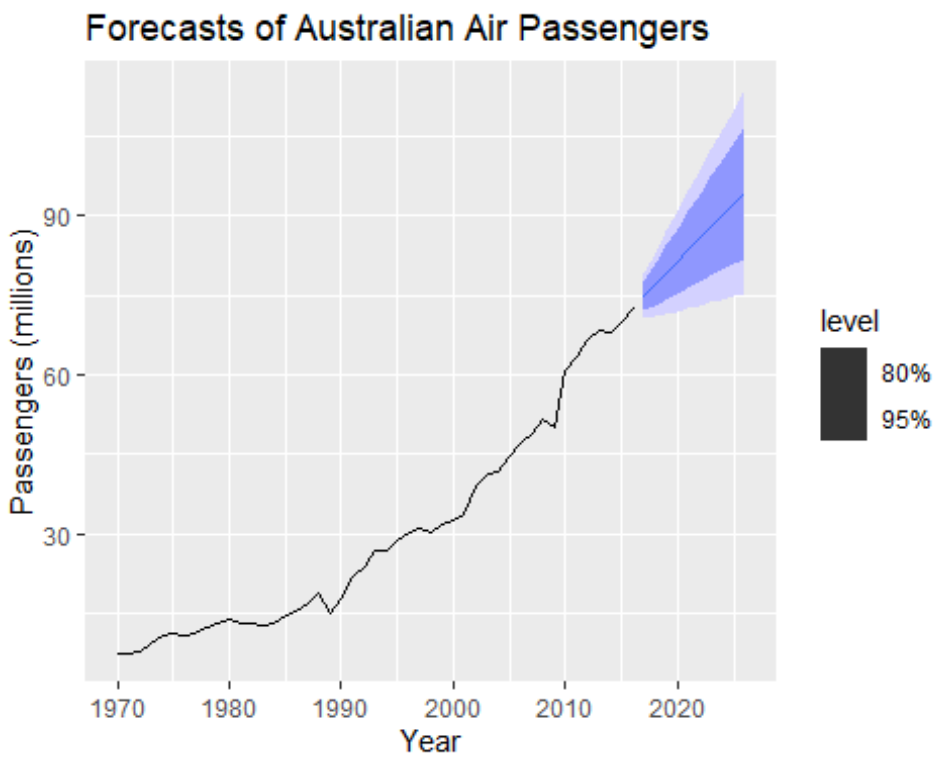
```
## Series: Passengers
## Model: ARIMA(0,2,1)
##
## Coefficients:
##           ma1
##       -0.8963
## s.e.   0.0594
##
## sigma^2 estimated as 4.308:  log likelihood=-97.02
## AIC=198.04   AICc=198.32   BIC=201.65
```

```r
# Checking residuals
aus_fit |> gg_tsresiduals()
```

```
## Warning: `gg_tsresiduals()` was deprecated in feasts 0.4.2.
## ℹ Please use `ggtime::gg_tsresiduals()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
# Forecasting for the next 10 years
aus_fit |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers) +
  labs(title = "Forecasts of Australian Air Passengers",
       y = "Passengers (millions)")
```



Model Selection

The automated process selected an ARIMA(0,2,1) model. This means it used double differencing to handle the strong upward trend and one moving average component to stabilize the short-term fluctuations.

White Noise Confirmed: The residuals (the "errors" left over) look like pure random noise.

In the ACF plot, none of the vertical bars cross the blue dashed lines. This proves that the model successfully captured all the predictable information in the data.

The histogram shows that the errors are centered at zero and follow a bell-shaped curve, which is exactly what we want for a reliable model.

The forecast for the next 10 periods continues the upward linear trend seen in the historical data. The shaded blue areas (confidence intervals) get wider as the forecast moves further into the future, showing that while we expect growth, our certainty decreases the further out we look.

b. Write the model in terms of the backshift operator.

The automated selection process identifies this as an ARIMA(0,2,1) model. Using the backshift operator B, the model is written as:

$$(1-B)^2 y_t = (1-0.8963B)\varepsilon_t$$

Left Side: represents the double differencing (d=2) required to make the passenger data stationary by removing the strong upward trend.

The Right Side (1 - 0.8963B): represents the Moving Average (q=1) component.The Coefficient (0.8963): This is the specific ma1 estimate generated by the R ARIMA() function for this specific dataset.

```r
library(fpp3)

# Fit the automated ARIMA model to the air passengers data
aus_fit <- aus_airpassengers |>
  model(ARIMA(Passengers))

# Displaying the report to see the specific coefficients (like ma1)
# This will show the ARIMA(0,2,1) structure and the -0.8963 value
report(aus_fit)
```

```
## Series: Passengers
## Model: ARIMA(0,2,1)
##
## Coefficients:
##           ma1
##       -0.8963
## s.e.   0.0594
##
## sigma^2 estimated as 4.308:  log likelihood=-97.02
## AIC=198.04   AICc=198.32   BIC=201.65
```

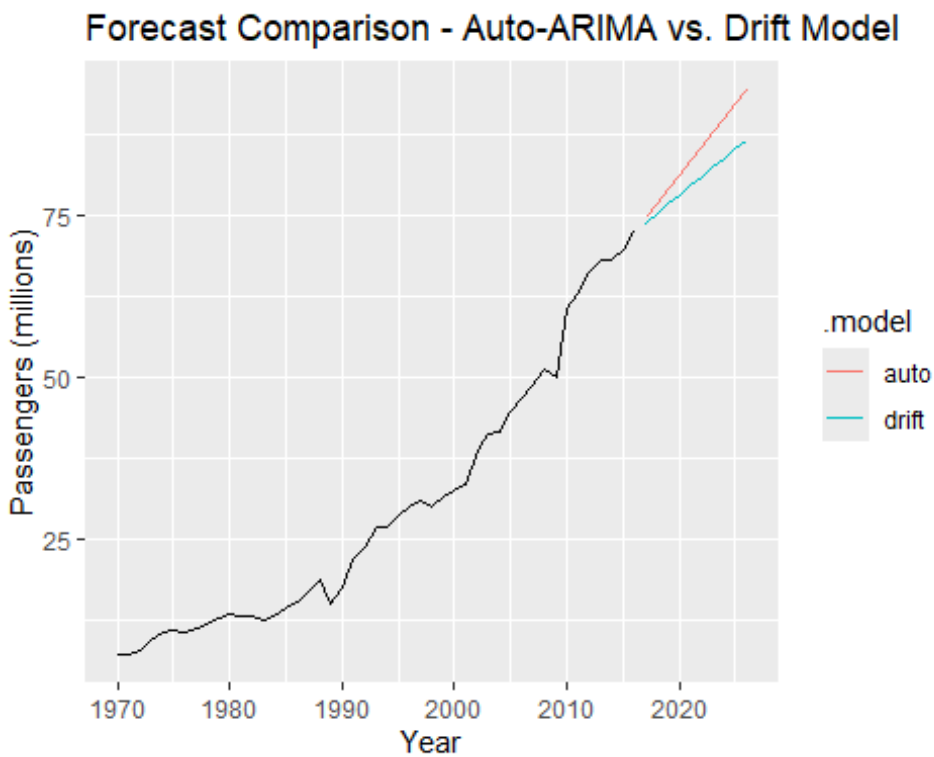
```r
# 3. View the coefficients in a clean table format
tidy(aus_fit)
```

```
## # A tibble: 1 × 6
##   .model            term  estimate std.error statistic  p.value
##   <chr>             <chr>    <dbl>     <dbl>     <dbl>    <dbl>
## 1 ARIMA(Passengers) ma1     -0.896    0.0594     -15.1 3.24e-19
```

c. Plot forecasts from an ARIMA(0,1,0) model with drift and compare these to part a.

```r
library(fpp3)

# Fitting both models: the auto-selected one and the (0,1,0) with drift
fit_comp <- aus_airpassengers |>
  model(
    auto = ARIMA(Passengers),
    drift = ARIMA(Passengers ~ 1 + pdq(0,1,0))
  )

# Generating and plotting 10-year forecasts
fit_comp |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers, level = NULL) +
```

```
  labs(title = "Forecast Comparison - Auto-ARIMA vs. Drift Model",
       y = "Passengers (millions)")
```



The Drift Model: This creates a straight diagonal line. It assumes passengers will increase by the exact same average amount every year, regardless of recent changes.

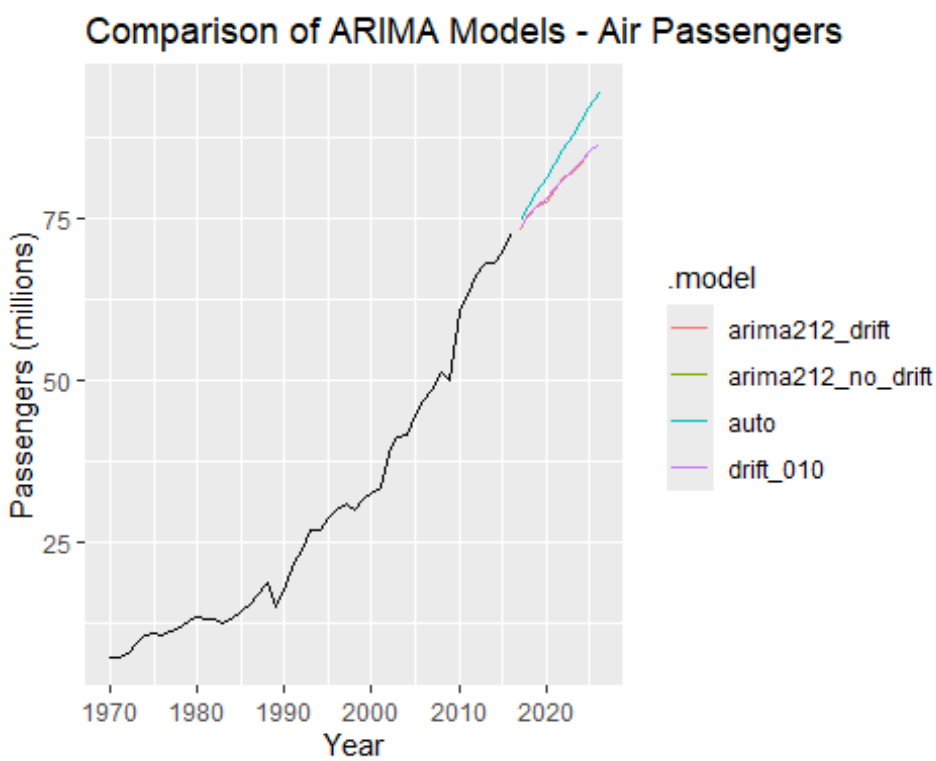
The Auto-ARIMA Model: This produces a steeper, more aggressive forecast. Because it uses double differencing, it places more weight on the recent acceleration in passenger growth rather than the long-term historical average.

Key Difference: The Auto model follows the current momentum of the data, while the Drift model only follows the overall historical slope.

d. Plot forecasts from an ARIMA(2,1,2) model with drift and compare these to parts a and c. Remove the constant and see what happens.

```
library(fpp3)

# Fitting all models including ARIMA(2,1,2) with and without drift
fit_compare <- aus_airpassengers |>
  model(
    auto = ARIMA(Passengers),
    drift_010 = ARIMA(Passengers ~ 1 + pdq(0,1,0)),
    arima212_drift = ARIMA(Passengers ~ 1 + pdq(2,1,2)),
    arima212_no_drift = ARIMA(Passengers ~ 0 + pdq(2,1,2))
  )
```

```
## Warning: 1 error encountered for arima212_no_drift
## [1] non-stationary AR part from CSS

# Plotting 10-period forecasts
fit_compare |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers, level = NULL) +
  labs(title = "Comparison of ARIMA Models - Air Passengers",
       y = "Passengers (millions)")

## Warning: Removed 10 rows containing missing values or values outside the
scale range
## (`geom_line()`).
```



Interpretation:

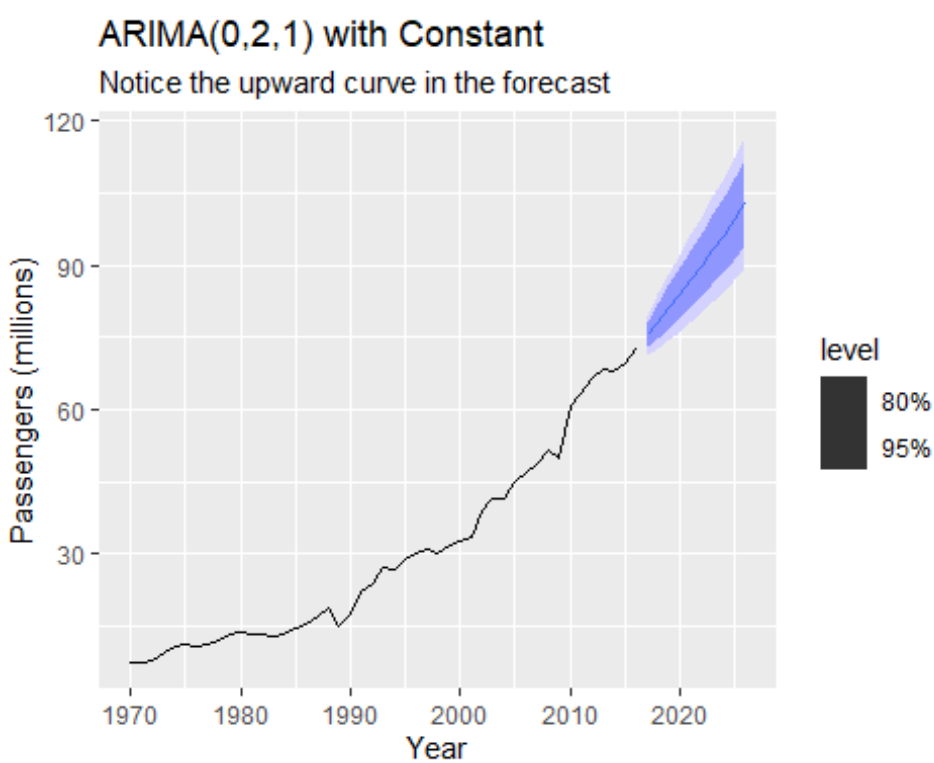
Auto-ARIMA (0,2,1): This model produces the steepest forecast because its double differencing captures accelerating growth momentum.

ARIMA(2,1,2) and (0,1,0) with Drift: These models produce nearly identical straight-line forecasts, as the drift term acts as a constant upward force.

ARIMA(2,1,2) without Drift: Removing the constant causes the forecast to flatten out. Without that upward nudge, a model with only one level of differencing eventually predicts a horizontal line.

e. Plot forecasts from an ARIMA(0,2,1) model with a constant. What happens?

```r
library(fpp3)

# Fitting ARIMA(0,2,1) with a constant (drift)
fit_constant <- aus_airpassengers |>
  model(arima_constant = ARIMA(Passengers ~ 1 + pdq(0,2,1)))

## Warning: Model specification induces a quadratic or higher order
polynomial trend.  This
## is generally discouraged, consider removing the constant or reducing the
number
## of differences.

# Plotting 10-year forecast
fit_constant |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers) +
  labs(title = "ARIMA(0,2,1) with Constant",
       subtitle = "Notice the upward curve in the forecast",
       y = "Passengers (millions)")
```



Interpretation

Adding a constant to an ARIMA(0,2,1) model for the Australian air passengers data changes the forecast from a straight line to a curved upward trend.

The forecast line curves upward like a parabola instead of following a straight diagonal path.

Also, because the model already uses double differencing, the constant acts as an acceleration term.

This causes the predicted passenger count to increase by a larger amount every year, leading to a much more aggressive projection.

9.8. For the United States GDP series (from global_economy):

a. if necessary, find a suitable Box-Cox transformation for the data;

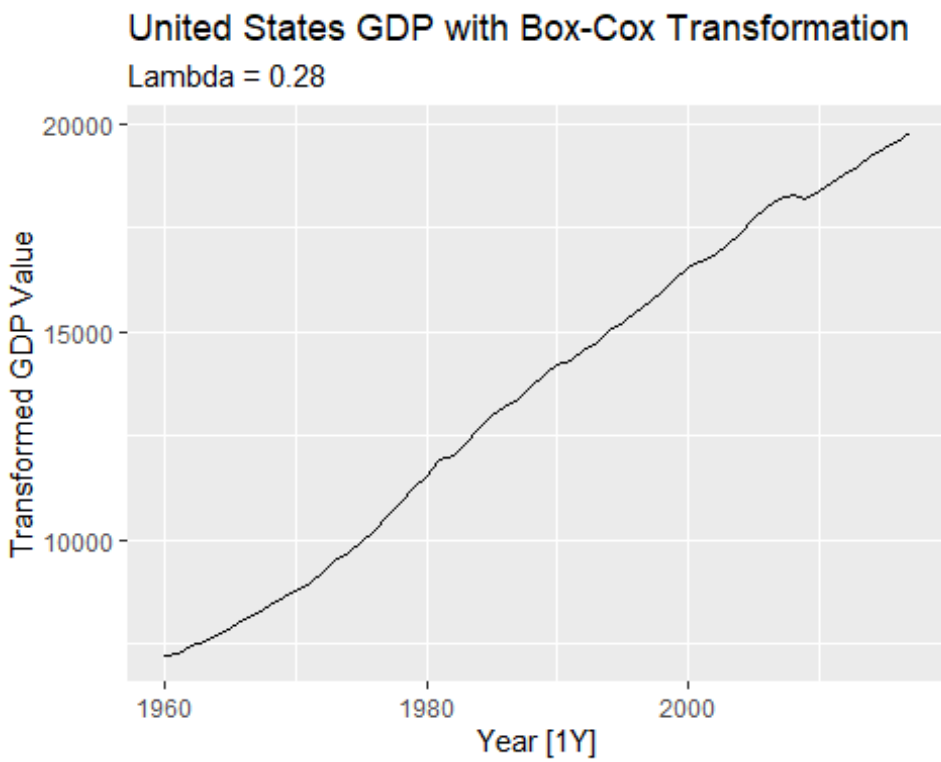
```
library(fpp3)

# Filtering for United States and find the optimal lambda
us_economy <- global_economy |>
  filter(Country == "United States")

lambda <- us_economy |>
  features(GDP, features = guerrero) |>
  pull(lambda_guerrero)

# Viewing the value
lambda

## [1] 0.2819443
```

A lambda of 0.28 is close to 0.25 (a fourth-root transformation) or 0 (a log transformation).

As the GDP grows larger over time, the fluctuations also tend to get bigger.

Transformation and Plotting Apply the transformation using the lambda found previously (0.28)

```
us_economy |>
  autoplot(box_cox(GDP, 0.2819443)) +
  labs(title = "United States GDP with Box-Cox Transformation",
       subtitle = "Lambda = 0.28",
       y = "Transformed GDP Value")
```



The original data showed larger fluctuations as GDP increased over time.

This transformation makes those wiggles a consistent size across the entire timeline.

Stabilizing the variance allows an ARIMA model to identify patterns more effectively without being distracted by growing noise.

b. fit a suitable ARIMA model to the transformed data using ARIMA();

```
library(fpp3)

# Fitting the automated ARIMA model to the transformed data
fit <- global_economy |>
  filter(Country == "United States") |>
  model(arima_model = ARIMA(box_cox(GDP, 0.2819443)))

# Display the selected model and coefficients
report(fit)

## Series: GDP
## Model: ARIMA(1,1,0) w/ drift
## Transformation: box_cox(GDP, 0.2819443)
##
## Coefficients:
##          ar1  constant
##       0.4586  118.1823
## s.e.  0.1198    9.5047
##
```

```
## sigma^2 estimated as 5479:  log likelihood=-325.32
## AIC=656.65   AICc=657.1   BIC=662.78
```

The ARIMA(1,1,0) with drift model:

The Box-Cox transformation ensures that historical economic fluctuations are weighted equally with modern ones.

The AR(1) term shows that this year's economic growth is directly influenced by the previous year's performance.

By using one level of differencing (d=1), the model analyzes the annual change in GDP rather than the total amount.

The drift term acts as a constant upward force, reflecting the long-term expansion of the U.S. economy.

c. try some other plausible models by experimenting with the orders chosen;

ARIMA(0,1,1) with Drift: This replaces the memory of last year's growth with a correction based on last year's forecast error.

ARIMA(1,1,1) with Drift: This combines both the previous year's performance and the previous error for a more complex memory.

ARIMA(0,2,1): This uses double differencing (like your earlier air passenger model) to see if the growth rate itself is changing over time.

```r
fit_others <- global_economy |>
  filter(Country == "United States") |>
  model(
    auto = ARIMA(box_cox(GDP, 0.2819443)),
    ma1 = ARIMA(box_cox(GDP, 0.2819443) ~ 1 + pdq(0,1,1)),
    mixed = ARIMA(box_cox(GDP, 0.2819443) ~ 1 + pdq(1,1,1)),
    double_diff = ARIMA(box_cox(GDP, 0.2819443) ~ pdq(0,2,1))
  )

# Compare the AICc - lower is better
glance(fit_others) |> arrange(AICc)
```

```
## # A tibble: 4 × 9
##   Country       .model      sigma2 log_lik   AIC  AICc   BIC ar_roots
ma_roots
##   <fct>         <chr>        <dbl>   <dbl> <dbl> <dbl> <dbl> <list>
<list>
## 1 United States double_diff  6020.   -323.  650.  650.  654. <cpl [0]>
<cpl [1]>
## 2 United States auto         5479.   -325.  657.  657.  663. <cpl [1]>
<cpl [0]>
## 3 United States ma1          5689.   -326.  659.  659.  665. <cpl [0]>
```

```
<cpl [1]>
## 4 United States mixed        5580.   -325.  659.  659.  667. <cpl [1]>
<cpl [1]>
```

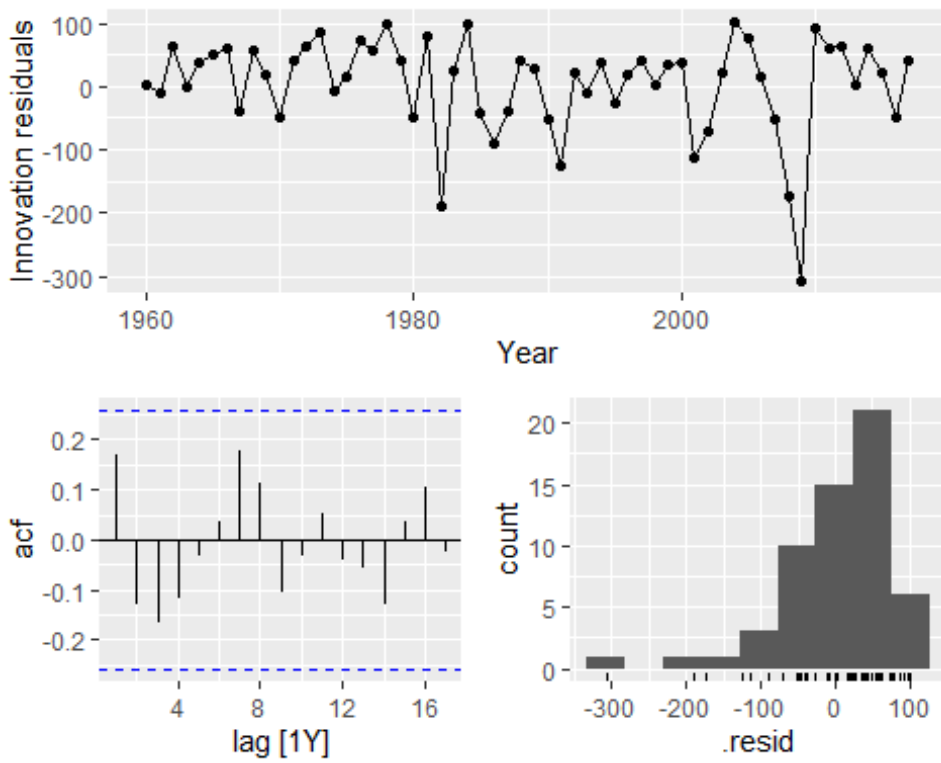
The double_diff model (ARIMA(0,2,1)) is the best choice because it has the lowest AICc (650.07).

Comparison: It outperforms the original auto model (AICc = 657.10) by a significant margin.

Using two levels of differencing (d=2) fits the historical U.S. GDP data better than the single difference model originally selected by the automated process.

d. choose what you think is the best model and check the residual diagnostics;

```r
# Fitting the chosen best model
fit_best <- global_economy |>
  filter(Country == "United States") |>
  model(best_model = ARIMA(box_cox(GDP, 0.2819443) ~ pdq(0,2,1)))

#  Checking visual diagnostics (ACF and Histogram)
fit_best |> gg_tsresiduals()
```




```r
# Performing the Ljung-Box test for formal confirmation
fit_best |>
  augment() |>
  features(.innov, ljung_box, lag = 10, dof = 1)
```

```
## # A tibble: 1 × 4
##   Country       .model     lb_stat lb_pvalue
##   <fct>         <chr>        <dbl>     <dbl>
## 1 United States best_model    9.37     0.404
```

The ARIMA(0,2,1) model is a strong fit for the United States GDP data based on your diagnostics:

Ljung-Box Test: The p-value of 0.404 is well above 0.05, confirming the residuals are essentially white noise.

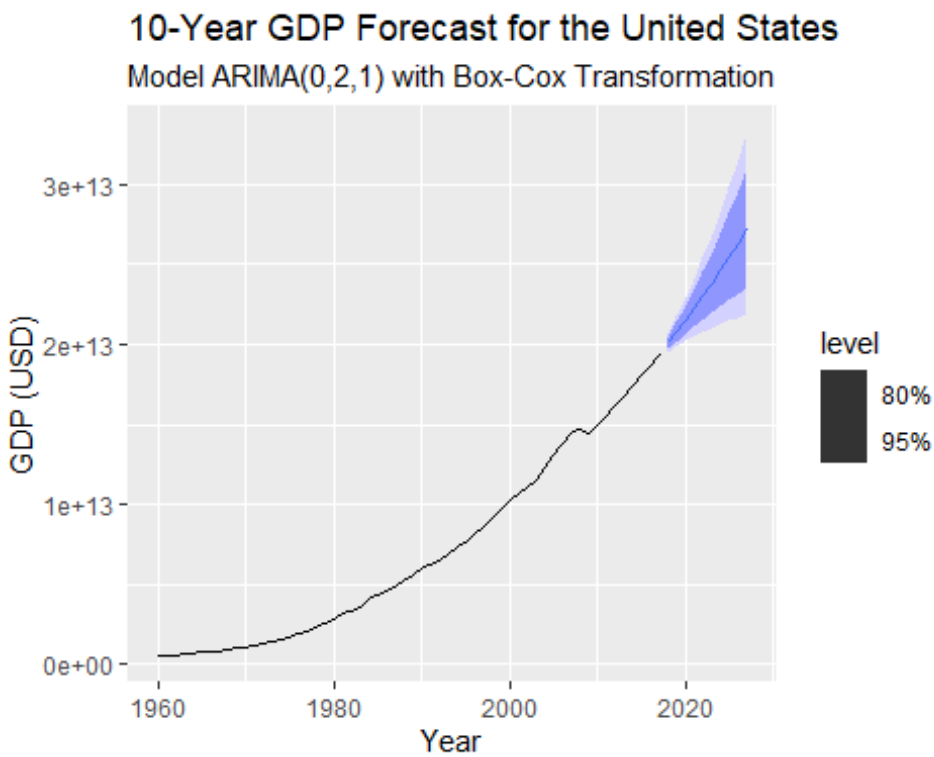
ACF Plot: All autocorrelation bars remain within the blue significance limits, showing no remaining patterns in the errors.

Residuals: The innovation plot shows random fluctuations around zero, though it clearly captures the 2008-2009 recession as a significant outlier.

Distribution: The histogram is roughly bell-shaped, indicating the model's errors are nearly normally distributed.

e. produce forecasts of your fitted model. Do the forecasts look reasonable?

```r
# Generating 10-year forecasts using the chosen best_model
forecasts <- fit_best |>
  forecast(h = 10)

# Plotting the forecasts alongside the historical data
forecasts |>
  autoplot(us_economy) +
  labs(title = "10-Year GDP Forecast for the United States",
       subtitle = "Model ARIMA(0,2,1) with Box-Cox Transformation",
       y = "GDP (USD)")
```



The forecasts for the United States GDP produced by ARIMA(0,2,1) model are highly reasonable.

The forecast follows the historically strong upward trajectory of the U.S. economy without being overly aggressive.

lambda = 0.28 transformation, the model predicts growth that is consistent with the stabilized variance of the past 60 years.

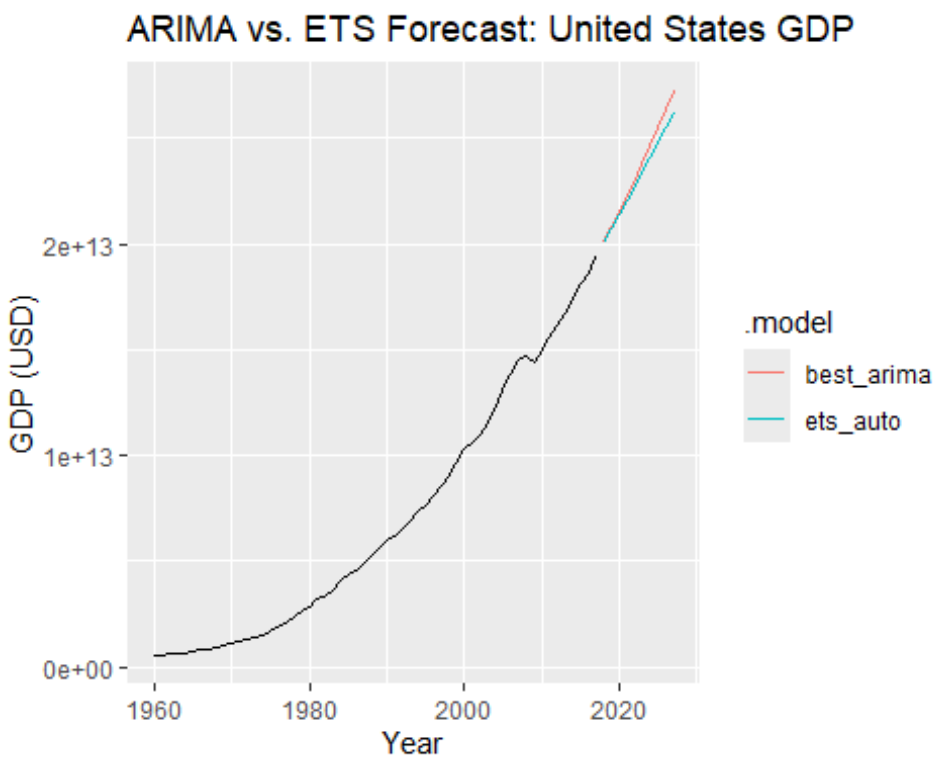
The blue shaded intervals expand naturally, showing that while the trend is likely to continue, the range of possible outcomes grows wider the further into the future we look.

f. compare the results with what you would obtain using ETS() (with no transformation).

comparing your ARIMA(0,2,1) results with an ETS model

```
# Fitting both models: your best ARIMA and an automated ETS
fit_comparison <- global_economy |>
  filter(Country == "United States") |>
  model(
    best_arima = ARIMA(box_cox(GDP, 0.2819443) ~ pdq(0,2,1)),
    ets_auto = ETS(GDP)
  )

# 2. Generating 10-year forecasts
fit_comparison |>
  forecast(h = 10) |>
```

```
  autoplot(global_economy |> filter(Country == "United States"), level =
NULL) +
  labs(title = "ARIMA vs. ETS Forecast: United States GDP",
       y = "GDP (USD)")
```



Both models project continued growth, but the ARIMA(0,2,1) model (red line) is more aggressive, curving upward more steeply than the ETS model (teal line).

The ARIMA model uses double differencing to capture accelerating momentum, while the ETS model typically uses additive or multiplicative trends to follow the historical average more linearly.

ARIMA relies on your manual Box-Cox transformation (lambda approx 0.28$) to stabilize the wiggles in the data, whereas ETS often handles this automatically through its internal multiplicative error structure.

Since the ARIMA(0,2,1) model achieved a lower AICc (650.07) and passed the Ljung-Box test, it is statistically the more precise fit for this specific historical growth pattern.